

Metrics

Prometheus · Datadog · Grafana – సెకనుకి 1.6 కోట్ల నమూనాలు ఒక సమస్య కాదు; అది సెకనుకి 56 MB. కష్టమంతా ఎన్ని వేర్వేరు విషయాలను కొలుస్తున్నారో అనేదానిలో ఉంది.

మూడు విరుపులు. మొదటిది – ఒక్క label కలిపితే **6 machines నుంచి 567 కి**. రెండోది – data ని కుంచిస్తే latency శిఖరం **2,496 ms నుంచి 281 ms** గా కనిపిస్తుంది. మూడోది – ప్రతి instance తన p99 చెబితే, వాటి సగటు నిజమైన p99 కంటే **85% తక్కువ**.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` `ls` `run` చేసినదే.

ఈ PROBLEM ని "ఒక TIME-SERIES DATABASE రాయ" అని చూస్తే అది సులభంగా అనిపిస్తుంది

కొలత: ఒక పేరు, కొన్ని labels, ఒక timestamp, ఒక సంఖ్య. కూడు, నిల్వ చెయ్యి, చూపించు. §1 lo కొలుస్తాం – ingestion సెకనుకి 56 MB, ఒక్క machine చేయగలదు.

నిజమైన కష్టం మూడు చోట్ల ఉంది, మరియు మూడూ మీరు ఏమి కొలుస్తున్నారో అనేదాని గురించి:

§4 – "ఏ label పెట్టాలి" అనేది ఒక storage నిర్ణయం, ఒక సౌలభ్య నిర్ణయం కాదు.

§7 – "ఎంతకాలం ఉంచాలి" అనేది "ఏ ప్రశ్నలు అడగగలం" అని నిర్ణయిస్తుంది.

§10 – "p99" అనే సంఖ్యని కలపలేము, మరియు చాలామంది దాన్ని కలుపుతూనే ఉంటారు.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: cardinality

3. Step – labels కలుపుదాం
4. మొదటి విరుపు – ఒక్క label, 6 machines నుంచి 567 కి
5. Step – కఠినమైన పరిమితి · మరియు దేన్ని label చేయకూడదు

Part 3 – రెండో విరుపు: నిల్వ మరియు కుంచిపు

6. Step – ప్రతి నమూనానీ ఉంచడం
7. రెండో విరుపు – 22 TB/రోజు · మరియు కుంచిస్తే ఏమి పోతుంది
8. Step – Gorilla · downsampling · మరియు దాని నిజమైన ఖర్చు

Part 4 – మూడో విరుపు: p99 కూడబడదు

9. Step – ప్రతి instance తన p99 చెబుతుంది
10. మూడో విరుపు – p99 ల సగటు p99 కాదు
11. Step – histogram · మరియు bucket హద్దుల ధర

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code · 17 లక్షల నమూనాలు · mutation testing
13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
14. నోటీస్ చెప్పాల్సిన English script

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు

200 services, ఒక్కొక్కటి 50 instances. ప్రతి 15 సెకన్లకీ వాటి metrics సేకరించాలి. Dashboards, alerts, మరియు "నిన్న రాత్రి 2 గంటలకి ఏమైంది" అనే ప్రశ్నకి జవాబు.

§4 లో వివరంగా లెక్కిస్తాం, కానీ ముందుగా మొత్తం: **24.19 కోట్ల time series**, 15 సెకన్ల విరామంతో.

24.19 కోట్ల series ÷ 15 సెకన్లు = సెకనుకి 1.6 కోట్ల నమూనాలు
ఒక్కో నమూనా కుంచించాక 3.47 bytes (§8 లో కొలిచాను)
→ సెకనుకి 56 MB

సెకనుకి 56 MB – ఇది ఒక సమస్య కాదు

ఒక సాధారణ SSD సెకనుకి 2 GB రాయగలదు (HLD 13 లో ఈ laptop మీద కొలిచాను – 2,138 MB/s).

కాబట్టి **ingestion** ఈ problem యొక్క కష్టమైన భాగం కాదు. మరియు అదే చాలామంది interview లో చర్చించే భాగం.

కష్టమైనవి మూడు, మరియు మూడూ ఆ "24.19 కోట్ల series" అనే సంఖ్య **ఎలా వచ్చింది** అనేదాని గురించి – మరియు **అది ఎంత సులభంగా 1,000 రెట్లు అవుతుంది** అనేదాని గురించి.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఎన్ని distinct time series ?	ఇదే ఏకైక ముఖ్యమైన scale సంఖ్య – §4
ఏ labels వాడతారు – మరియు వాటి cardinality?	§4. <code>user_id</code> ఒక్కటి మొత్తం system ని చంపుతుంది
ఎంతకాలం ఉంచాలి – మరియు ఏ resolution లో?	§8 – మరియు resolution తగ్గిస్తే కొన్ని ప్రశ్నలు అసాధ్యమవుతాయి

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Percentiles కావాలా?	అవును అంటే §11 – మరియు అది client వైపు design ని మారుస్తుంది
Metrics push చేస్తారా, pull చేస్తారా?	Pull అయితే "service పోయింది" అనేది కూడా ఒక సంకేతం
ఒక నమూనా పోతే ఫరవాలేదా?	Metrics కి సాధారణంగా అవును – ఇది HLD 17 కి వ్యతిరేకం
Alerts ఈ data మీదేనా?	అవును అంటే query latency ఒక availability ప్రశ్న
ఎంత వెనక్కి వెళ్ళి అడగాలి?	ఒక గంటా, ఒక సంవత్సరమా – నిల్వ వ్యూహం పూర్తిగా వేరు

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"ఏ label కి అత్యధిక cardinality ఉంది – మరియు అది ఎవరు నిర్ణయిస్తారు?"

ఎందుకంటే §4 lo కొలుస్తాం: ఒక్క label, 100 విలువలతో, మే system ని **6 machines నుంచి 567** కి తీసుకెళ్తుంది.

మరియు ఆ label ని జోడించేది మరు కాదు – ఒక application developer, ఒక సాధారణ pull request lo, "debugging కి ఉపయోగపడుతుంది" అనే సరైన కారణంతో.

కాబట్టి ఇది ఒక **architecture ప్రశ్న కాదు – ఒక పరిపాలన ప్రశ్న**. మరియు దానికి జవాబు code lo ఒక కఠినమైన పరిమితి కావాలి (§5).

Part 2 – మొదటి విరుపు: cardinality

3. Step – labels కలుపుదాం

Metrics యొక్క మొత్తం శక్తి labels లో ఉంది. `http_requests_total` ఒక్క సంఖ్య – ఉపయోగం తక్కువ. Labels తో అది ఒక విశ్లేషణ సాధనం అవుతుంది:

```
http_requests_total{service="checkout", instance="pod-7", endpoint="/pay",
method="POST", status="500"}
```

ఇప్పుడు "checkout లో /pay మీద POST లకి 500 లు పెరిగాయో?" అని అడగగలరు. ఇది సరిగ్గా సరైన design.

కానీ ప్రతి label ఒక గుణకారం.

4. మొదటి విరుపు – ఒక్క label, 6 machines నుంచి 567 కి

ఒక service · ఒక metric: `http_requests_total{...}`

కలిపిన label	విలువలు	ఆ metric యొక్క series	memory
instance	50	50	150 KB
endpoint	40	2,000	6 MB
method	5	10,000	30 MB
status	12	1,20,000	360 MB

(ఇది ఒక పై-పరిమితి – అన్ని కలయికలూ నిజంగా జరగవు. కానీ గుణకారమే ఇక్కడ పాఠం.)

ఒక service: 10 metrics పూర్తి labels తో + 190 సాధారణమైనవి
→ 12,09,500 series/service × 200 services = 24,19,00,000 series
→ 725.7 GB (ఇది పెద్దది, కానీ నిర్వహించదగినది)

726 GB – 6 machines. పెద్దదే, కానీ ఇది నడుస్తుంది. ఇప్పుడు ఒక developer ఒక label జోడిస్తాడు:

ఇప్పుడు ఒక్క label కలుపుదాం – "ఏ user" (debugging కి ఉపయోగం అనిపిస్తుంది):

user_id కి విలువలు	మొత్తం series	memory	ఎన్ని machines (128 GB)
label లేదు	24,19,00,000	725.7 GB	6
100	24,19,00,00,000	72.6 TB	567
10,000	24,19,00,00,00,000	7.3 PB	56,696
10,00,000	24,19,00,00,00,00,000	725.7 PB	6 మిలియన్

ఒక్క LABEL · 100 విలువలు · 6 MACHINES → 567

మరియు ఆ label ని జోడించిన వ్యక్తి తప్పు ఏమీ చేయలేదు. "ఏ user కి errors వస్తున్నాయో చూడాలి" అనేది ఒక సహేతుకమైన కోరిక.

సమస్య ఏమిటంటే metrics system lo ఒక label జోడించడం ఒక్క పంక్తి, మరియు దాని ఖర్చు ఎక్కడా కనిపించదు – ఒక వారం తర్వాత memory నిండి Prometheus చనిపోయే వరకు.

మరియు REQUEST_ID అయితే ఇది ఒక సంఖ్య కాదు, ఒక ధార

user_id కి ఒక పరిమితి ఉంది – మీకు ఎంతమంది users ఉంటే అంతే.

request_id ప్రతి request కీ ప్రత్యేకం. సెకనుకి 10 లక్షల requests అంటే సెకనుకి 10 లక్షల కొత్త series, మరియు అవి ఎప్పటికీ పునరావృతం కావు.

అది ఒక cardinality సమస్య కాదు – అది metrics system ని ఒక log system గా వాడటం. మరియు రెండూ వేర్వేరు కారణాలకి ఉన్నాయి (§15).

5. Step – కఠినమైన పరిమితి · మరియు దేన్ని label చేయకూడదు

పరిష్కారం ఒక documentation పంక్తి కాదు – code lo ఒక కఠినమైన పరిమితి:

```
observe(name, labels, v) {
  const k = Registry.key(name, labels);
  let h = this.#s.get(k);
  if (!h) {
    if (this.#s.size >= this.maxSeries) { this.stats.rejected++; return false; } //
    ← కఠినమైన పరిమితి
    h = new Histogram(this.bounds); this.#s.set(k, h); this.stats.created++;
  }
  h.observe(v); return true;
}
```

ఒక కొత్త SERIES ని తిరస్కరించడం ఎందుకు సరైనది

ప్రత్యామ్నాయం – పరిమితి లేకపోవడం – అంటే ఒక తప్పు label మీ మొత్తం monitoring ని కూల్చేస్తుంది. మరియు monitoring కూలినప్పుడు మీకు అది కూలించని కూడా తెలియదు.

కొత్త series ని తిరస్కరిస్తే, మీరు కొన్ని data పోగొట్టుకుంటారు మరియు మిగతాదంతా నిలబెట్టుకుంటారు. మరియు ఆ తిరస్కరణ సంఖ్యే ఒక metric – అది మీకు ఎవరో ఒక చెడ్డ label జోడించారని చెబుతుంది.

నా fuzz lo 17.5 లక్షల నమూనాల్లో 51,596 తిరస్కరించబడ్డాయి – ఆ దారి నిజంగా పరీక్షించబడింది.

ఏది label చేయకూడదు అనే ఆచరణాత్మక నియమం:

Label	సరేనా?	ఎందుకు
status , method , endpoint	అవును	పరిమిత, తెలిసిన, చిన్న సమితి
instance , region , version	అవును	పెరుగుతుంది కానీ నెమ్మదిగా, మరియు మీ నియంత్రణలో
user_id , customer_id	కాదు	మీ వ్యాపారంతో పాటు పెరుగుతుంది – §4
request_id , trace_id	ఎప్పటికీ కాదు	అపరిమితం. ఇది logs/traces పని – §15
error_message	కాదు	ఒక free-text field. cardinality అనూహ్యం
url (query string తో)	కాదు	?t=1699... ప్రతిసారీ కొత్తది

Part 3 – రెండో విరుపు: నిల్వ మరియు కుంచింపు

6. Step – ప్రతి నమూనానీ ఉంచడం

ఒక నమూనా = ఒక timestamp (8 bytes) + ఒక విలువ (8 bytes) = **16 bytes**.

24.19 కోట్ల series × రోజుకి 5,760 నమూనాలు × 16 bytes = 22 TB/రోజు

సంవత్సరానికి **8 PB**. ఒక్క metrics system కోసం.

7. రెండో విరుపు – 22 TB/రోజు · మరియు కుంచిస్తే ఏమి పోతుంది

Time series data చాలా కుంచింపదగినది, ఎందుకంటే అది నెమ్మదిగా మారుతుంది. **Gorilla** (Facebook యొక్క పద్ధతి) రెండు ఉపాయాలు వాడుతుంది:

- **Timestamps:** delta-of-delta. ప్రతి 15 సెకన్లకీ నమూనా అంటే delta ఎప్పుడూ 15 – కాబట్టి delta-of-delta సున్నా, అంటే ఒక్క bit.
- **Values:** ముందటి విలువతో XOR. నెమ్మదిగా మారే గణాంకాలకి XOR lo చాలా సున్నాలు ఉంటాయి.

దాన్ని అమలు చేసి కొలిచాను:

5,760 నమూనాలు (ఒక రోజు, 15 సె విరామం) · ముడి = 16 bytes/నమూనా

series రకం	ముడి	Gorilla	bytes/నమూనా	కుంచింపు
మారని విలువ (42)	90 KB	1.4 KB	0.25	63.4×
నెమ్మదిగా మారే gauge	90 KB	29.9 KB	5.31	3.0×
పెరుగుతున్న counter	90 KB	13.1 KB	2.32	6.9×
అప్పుడప్పుడూ శిఖరాలు	90 KB	33.6 KB	5.98	2.7×

సగటు 3.47 BYTES/సమూహం - మరియు నిజమైన GORILLA దీని కంటే మెరుగు

Facebook యొక్క paper **1.37 bytes/సమూహం** నివేదించింది. నా సంఖ్య రెట్టింపుకి పైగా ఉంది, మరియు కారణం నా **కృత్రిమ data**: నా "gauge" ప్రతి సమూహాకీ యాదృచ్ఛికంగా మారుతుంది.

నిజమైన metrics అంత చలించవు - చాలా gauges గంటల తరబడి **సరిగ్గా అదే విలువ** ఉంటాయి, మరియు అప్పుడు XOR సున్నా = ఒక్క bit. అదే మొదటి వరుస (63.4×) చూపిస్తోంది.

కాబట్టి 3.47 ఒక **నిరాశావాద** అంచనా, మరియు నేను దాన్నే వాడుతున్నాను.

24.19 కోట్ల series × రోజుకి 5,760 సమూహాలు:

ముడి : 22 TB/రోజు

Gorilla : 4.8 TB/రోజు

సంవత్సరానికి : 1.8 PB

8 PB → **1.8 PB**. మెరుగు, మరియు ఇంకా ఎక్కువ. కాబట్టి తర్వాతి అడుగు: **పాత data ని కుంచించడం** - 15 సెకన్ల సమూహాలని 5 నిమిషాల సారాంశాలుగా.

ఇక్కడే విరుపు ఉంది.

8. Step - Gorilla · downsampling · మరియు దాని నిజమైన ఖర్చు

ఒక రోజు latency data, అందులో అప్పుడప్పుడూ 30-సెకన్ల శిఖరాలు. 20 సమూహాలను (5 నిమిషాలు) ఒక ముద్దగా కుంచిస్తే:

ఏ ప్రశ్న	ముడి data నుంచి	avg మాత్రమే ఉంచితే	avg+max ఉంచితే
గరిష్ట latency	2496	281	2496
సగటు latency	57	57	57
p99 latency	60	278	2420

మొదటి వరుస - శిఖరం 2,496 MS, కానీ 281 MS గా కనిపిస్తుంది

30 సెకన్ల శిఖరం 5 నిమిషాల సగటులో **8.9 రెట్లు** కరిగిపోయింది.

అంటే: ఒక నెల క్రితం జరిగిన ఒక outage ని మీరు ఇప్పుడు చూస్తే, అది **జరగనట్లే** కనిపిస్తుంది. Data అక్కడే ఉంది - కానీ మీరు అడిగిన ప్రశ్నకే జవాబు ఇక అందులో లేదు.

సగటు ఒక కుంచించు కాదు - అది ఒక తొలగింపు.

రెండో వరుస చూపిస్తోంది సగటు మాత్రం **ఖచ్చితంగా బతికింది** (57 = 57), మరియు గరిష్టాన్ని విడిగా ఉంచితే అది కూడా బతుకుతుంది (2496 = 2496).

కానీ మూడో వరుస - p99 - రెండు విధాలుగానూ విరిగింది:

నిజమైన P99 60 MS • కుంచిచాక 278 లేదా 2,420

ఎందుకంటే p99 అంటే "1% నమూనాలు దీని కంటే ఎక్కువ". ముడి data lo 5,760 నమూనాలు ఉన్నాయి, శిఖరాలు 18 – అంటే 0.3%, p99 కంటే పైన.

కుంచిచాక 288 ముద్దలు మాత్రమే, శిఖరాలు 9 ముద్దల్లో ఉన్నాయి – అంటే 3%, ఇప్పుడు p99 కంటే కింద.

మీరు నమూనాల సంఖ్యని మార్చారు, కాబట్టి "1%" అనే మాట వేరే విషయాన్ని సూచిస్తోంది.

మరియు ఇది max ని ఉంచినా పరిష్కారం కాదు – 2,420 అనేది 60 కంటే 40 రెట్లు ఎక్కువ. **Percentiles ని సారాంశాల నుంచి తిరిగి లెక్కించలేరు.** వాటిని కొలిచే సమయంలోనే ఉంచాలి – అదే §11.

Part 4 – మూడో విరుపు: p99 కూడబడదు

9. Step – ప్రతి instance తన p99 చెబుతుంది

50 instances. ఒక్కొక్కటి తన latency లను కొలిచి, ప్రతి 15 సెకన్లకీ ఒక సంఖ్య పంపుతుంది: **నా p99**.

సర్వర్ వైపు వాటిని కలపాలి. సహజమైన రెండు ఎంపికలు: **సగటు లేదా గరిష్ఠం**.

రెండూ తప్పు, మరియు ఏది "తక్కువ తప్పు" అనేది **పరిస్థితిని బట్టి మారుతుంది** – అదే అసలు సమస్య.

10. మూడో విరుపు – p99 ల సగటు p99 కాదు

పరిస్థితి	నిజమైన p99	p99 ల సగటు	p99 ల గరిష్ఠం
50 instances సమానం • ఒకటి 10 రెట్లు నెమ్మది	400 ms	349 (-13%)	2829 (+608%)
50 instances • ఒకటి 90% traffic మోస్తుంది (నెమ్మదిగా)	2204 ms	329 (-85%)	2255 (+2%)
50 instances అన్నీ ఒకేలా	300 ms	298 (-1%)	323 (+8%)

మొదటి రెండు వరుసల్లో విశేష మారిపోయింది

వరుస 1 – సగటు 13% తక్కువ, గరిష్ఠం **608% ఎక్కువ**. ఇక్కడ సగటు మెరుగు.

వరుస 2 – సగటు **85% తక్కువ**, గరిష్ఠం 2% ఎక్కువ. ఇక్కడ గరిష్ఠం మెరుగు.

అంటే **"p99 లని ఎలా కలపాలి" అనే ప్రశ్నకి స్థిరమైన జవాబు లేదు** – ఎందుకంటే జవాబు మీకు తెలియని దాని మీద (traffic ఎలా పంచబడింది) ఆధారపడుతుంది.

మరియు మూడో వరుస ప్రమాదకరమైనది: అన్నీ ఒకేలా ఉంటే సగటు కేవలం **1% తప్పు**. అంటే మీ dashboard **సాధారణ రోజుల్లో సరిగ్గా** కనిపిస్తుంది, మరియు **ఏదో తప్పు జరిగిన రోజున మాత్రమే** అబద్ధం చెబుతుంది.

ఇది HLD 17 యొక్క అదే ఆకారం - మరియు అదే ఒక సూచన

HLD Deep 17 §10 lo: ప్రత్యేక users సంఖ్యలను కలపలేము (shards కలిపితే 405% ఉబ్బు).

ఇక్కడ: p99 లను కలపలేము.

రెండూ ఒకే కారణం - ఆ సంఖ్య ఒక సారాంశం, మరియు సారాంశాల మీద మీరు అసలు ఆపరేషన్ ని తిరిగి చేయలేరు. Count కూడబడుతుంది ఎందుకంటే అది ఒక మొత్తం. p99 కూడబడదు ఎందుకంటే అది ఒక క్రమ స్థితి (order statistic).

పరిష్కారం రెండు చోట్లా ఒకటే: సారాంశం కాదు, కలపదగిన నిర్మాణం పంపడం. అక్కడ HLL. ఇక్కడ histogram.

11. Step – histogram • మరియు bucket హద్దుల ధర

ప్రతి instance ఒక సంఖ్య కాదు - ఒక histogram పంపుతుంది: "1 ms కంటే తక్కువ ఎన్ని, 2 ms కంటే తక్కువ ఎన్ని, 4 ms కంటే తక్కువ ఎన్ని...".

```
merge(o) {  
  for (let i = 0; i < this.counts.length; i++) this.counts[i] += o.counts[i]; // ←  
  కూడిక  
  this.sum += o.sum; this.n += o.n;  
  if (o.min < this.min) this.min = o.min;  
  if (o.max > this.max) this.max = o.max;  
}
```

HLL గరిష్టం తో కలుస్తుంది. HISTOGRAM కూడిక తో. ఆ తేడా యాదృచ్ఛికం కాదు

HLL యొక్క ప్రతి register "ఈ నమూనాని చూశానా" అని గుర్తుపెడుతుంది - ఒకే user రెండుసార్లు వచ్చినా జవాబు మారకూడదు. కాబట్టి గరిష్టం (idempotent).

Histogram యొక్క ప్రతి bucket "ఎన్నిసార్లు" అని గుర్తుపెడుతుంది - ప్రతి సంఘటనా లెక్కలోకి రావాలి. కాబట్టి కూడిక.

రెండూ merge అవుతాయి, కానీ అవి గుర్తుపెట్టే విషయాన్ని బట్టి వేరే ఆపరేషన్ తో. దీన్ని తారుమారు చేస్తే §12 lo mutation testing దాన్ని 100/100 పట్టుకుంటుంది.

ఇది ఉచితం కాదు. Histogram యొక్క జవాబు ఎప్పుడూ ఒక bucket హద్దు - ఖచ్చితమైన విలువ కాదు:

buckets	ఒక్కో bucket నిష్పత్తి	histogram నుంచి p99	తప్పు %
8	4×	1024	151.5%
16	2×	512	25.7%
32	1.5×	438	7.5%
64	1.2×	410	0.7%

నిజమైన p99 = 407 ms

మరియు ఇప్పుడు §4 తిరిగి వచ్చింది

64 buckets తప్పుని 0.7% కి తెస్తాయి. కానీ **ఒక histogram = అన్ని buckets కీ ఒక్కో time series.**

అంటే `http\_latency` ని histogram గా మారిస్తే మే series సంఖ్య **64 రెట్లు** అవుతుంది – §4 యొక్క అదే గుణకారం, ఇప్పుడు మీరే కావాలని చేస్తున్నారు.

కాబట్టి సరైన ఎంపిక: **ముఖ్యమైన కొన్ని metrics కి histogram** (సూక్ష్మమైన buckets తో), మిగతా వాటికి సగటు మరియు గరిష్ఠం. మరియు bucket హద్దులని మే **SLO** చుట్టూ ఎంచుకోవడం – 200 ms SLO ఉంటే 150, 200, 250 దగ్గర దట్టంగా.

ఏకరీతి buckets ఎప్పుడూ తప్పు. మీకు ముఖ్యమైన చోట దట్టంగా, మిగతా చోట పలచగా.

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code • 17 లక్షల నమూనాలు • mutation testing

ఏడు నియమాలు: **cardinality** పరిమితి ఎప్పుడూ దాటకూడదు, **count** మరియు **sum** ఖచ్చితంగా కూడబడాలి, **min/max** ఖచ్చితం, **quantile** నిజమైన **rank [q·n]** కి ఒక ఎగువ హద్దు, ఒక హద్దు వరకు **cumulative count** నిజంతో సరిగ్గా సమానం, **buckets మొత్తం = n**, **merge క్రమం** మారినా అదే **counts**.

300 యాదృచ్ఛిక runs • 17,48,404 నమూనాలు
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

observed	17,48,404
rejected	51,596
series	28,751
merges	300
floatDrift	8,949

నా మొదటి **QUANTILE** నియమం **300 LO 205** విఫలమైంది – మరియు **CODE** సరైనదే

నేను histogram యొక్క **quantile(0.5)** ని ఒక **అంతర్వేశనం చేసిన percentile** తో పోల్చాను, మరియు అంచనా నిజం కంటే **0.09 ms తక్కువ** అని పదే పదే వచ్చింది.

కారణం: రెండూ వేర్వేరు **rank conventions**. Histogram "cumulative count q·n కి చేరిన మొదటి bucket" ని ఇస్తుంది – అంటే అది **rank [q·n]** వద్ద ఉన్న నమూనా కి ఒక ఎగువ హద్దు. అంతర్వేశనం చేసిన percentile **q·(n-1)** దగ్గర ఉంటుంది – ఒక rank తేడా.

"p99" అనే మాటకి కనీసం తొమ్మిది వేర్వేరు నిర్వచనాలు ఉన్నాయి, మరియు రెండు systems వేర్వేరు వాటిని వాడితే మీ dashboards ఒకదానితో ఒకటి ఏకీభవించవు – ఎవరూ తప్పు చేయకుండానే.

మరియు **FLOATDRIFT 8,949** – ఒక చిన్న కానీ నిజమైన విషయం

28,751 series ని రెండు వేర్వేరు క్రమాల్లో merge చేశాను. **Bucket counts ప్రతిసారీ ఖచ్చితంగా సమానం** (అవి పూర్ణ సంఖ్యలు).

కానీ **8,949 series lo sum** వేరుగా వచ్చింది – ఎందుకంటే float కూడిక **associative కాదు**. $(a+b)+c \neq a+(b+c)$.

అంటే మీ "మొత్తం latency" సంఖ్య **ఏ క్రమంలో shards జవాబిచ్చాయో** దాని మీద చివరి అంకెల్లో ఆధారపడుతుంది. Metrics కి ఇది ఫరవాలేదు. **HLD 17 యొక్క billing** కి ఇది ఫరవాలేదు కాదు – అందుకే అక్కడ counts ని integers గా ఉంచాలి.

==== mutation testing ====

మార్పు లేని code	→ 0/300 విఫలం
histogram merge lo కూడికకి బదులు గరిష్ఠం (HLL లాగా)	→ 100/100 విఫలం
quantile ఎగువ హద్దుకి బదులు దిగువ హద్దు ఇస్తే	→ 100/100 విఫలం
cardinality పరిమితి తీసేస్తే	→ 17/100 విఫలం
ఉదా: registry lo 80 series > పరిమితి 79	
bucket ఎంపికలో < ని <= గా మార్చిస్తే	→ 100/100 విఫలం
merge lo min/max ని నవీకరించకపోతే	→ 100/100 విఫలం
combine lo clone() కి బదులు నేరుగా వాడితే	→ 100/100 విఫలం

ఒక MUTATION మొదట బతికింది, మరియు అది నా DATA యొక్క లోపం

bucket ఎంపికలో < ని <= గా మార్చడం – అంటే సరిగ్గా ఒక హద్దు మీద ఉన్న విలువ ఏ bucket కి వెళ్తుంది అనేది. మొదట అది **0/100**.

కారణం: నా fuzz నిరంతర యాదృచ్ఛిక విలువలు ఉత్పత్తి చేస్తోంది, కాబట్టి "సరిగ్గా హద్దు మీద" ఎప్పుడూ జరగలేదు.

నిజ జీవితంలో అది **సర్వసాధారణం** – latency లు తరచుగా పూర్ణ మిల్లీసెకన్లు, మరియు bucket హద్దులు గుండ్రని సంఖ్యలు (1, 5, 10, 25, 50). కాబట్టి 25% నమూనాలని సరిగ్గా ఒక హద్దు మీద ఉంచాను – ఇప్పుడు **100/100**.

ఒక mutation బతికితే మూడు అవకాశాలు: బలహీనమైన పరీక్ష, చచ్చిన code, లేదా – ఇక్కడిలా – నిజ జీవితాన్ని ప్రతిబింబించని data.

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి	ఏ సంఖ్య చెప్పాలి
0-5	Requirements · ingestion సమస్య కాదు	సెకనుకి 1.6 కోట్ల నమూనాలు = 56 MB/s
5-14	Labels · cardinality	726 GB → ఒక్క label → 72.6 TB · 6 → 567 machines
14-18	కరినమైన పరిమితి · ఏది label చేయకూడదు	<code>request_id</code> = సెకనుకి 10 లక్షల కొత్త series
18-24	నిల్వ · Gorilla	16 → 3.47 bytes/సమూహం · 8 PB → 1.8 PB
24-32	Downsampling ఏమి పోగొడుతుంది	శిఖరం 2,496 → 281 · p99 తిరిగి లెక్కించలేము
32-40	p99 కూడబడదు	సగటు -85% , గరిష్ఠం +608% , ఏది మెరుగో మారుతుంది
40-45	Histogram · bucket ధర	64 buckets = 0.7% తప్పు = 64x series

ఈ PROBLEM LO మమ్మల్ని వేరుగా నిలబెట్టే ఒక్క క్షణం

చాలామంది "p99 కోసం histogram వాడతాను" అని చెప్పి ముందుకి వెళ్ళారు.

మీరు ఒక అడుగు ముందుకి: "p99 ని కలవలేము ఎందుకంటే అది ఒక order statistic, ఒక మొత్తం కాదు. నేను కొలిచినప్పుడు – ఒక instance 90% traffic మోసినప్పుడు p99 ల సగటు నిజం కంటే 85% తక్కువ ఇచ్చింది; ఒక instance నమ్మదైనప్పుడు గరిష్ఠం 608% ఎక్కువ ఇచ్చింది. ఏది తక్కువ తప్పు పరిస్థితిని బట్టి మారుతుంది, కాబట్టి స్థిరమైన జవాబు లేదు. Histogram పరిష్కరిస్తుంది ఎందుకంటే counts కూడబడతాయి. కానీ ఒక histogram = 64 time series, కాబట్టి నేను \$4 యొక్క cardinality problem ని కావాలని కొంటున్నాను – అందుకే దాన్ని కొన్ని metrics కి మాత్రమే."

ఇది ఒక పరిష్కారం చెప్పడం కాదు – ఆ పరిష్కారం ఏ ఖర్చుతో వస్తుందో మీకు తెలుసని చూపుతుంది.

14. నోటితో చెప్పాల్సిన English script

Framing గురించి:

"I'd start by getting the easy part out of the way. Two hundred services at fifty instances each is about two hundred forty million time series, which at a fifteen-second scrape is sixteen million samples a second. Compressed that's fifty-six megabytes a second – one machine. So ingestion isn't the hard part, and I'd rather spend the time on three things that are: how that series count is arrived at, what retention destroys, and why the number every service reports can't be combined."

Cardinality గురించి:

"Every label multiplies. The baseline came out at about seven hundred twenty-six gigabytes of index and active chunks – six machines, fine. Then someone adds a user_id label with a hundred values, for a perfectly good reason, and it's seventy-two terabytes and five hundred sixty-seven machines. The cost of that change is invisible at the point it's made – one line in a pull request – and it shows up a week later when Prometheus runs out of memory. So I'd enforce a hard series cap in the client that rejects new series and exports the rejection count as its own metric. And I'd treat request_id or trace_id as never-label: those aren't high cardinality, they're unbounded – a million new series a second – which means someone is using a metrics system as a log system."

Retention గురించి:

"Gorilla compression got me from sixteen bytes a sample to about three and a half – delta-of-delta on timestamps, XOR on values. Facebook's paper reports one point three seven, and mine is worse because my synthetic gauges jitter every sample where real ones sit still for hours, so I'd treat three point five as pessimistic. That takes eight petabytes a year to one point eight. Then you downsample, and this is where I'd slow down: I measured a thirty-second spike of two thousand five hundred milliseconds showing up as two hundred eighty-one after five-minute averaging – nine times understated. Averages don't compress a spike, they delete it. Keeping max alongside recovers the max exactly. But percentiles can't be recovered at all – my p99 went from sixty milliseconds raw to two hundred seventy-eight or two thousand four hundred depending on what I kept, because changing the number of samples changes what 'one percent' refers to."

Percentiles గురించి:

"Which leads to the last one. If every instance reports its own p99, there's no correct way to combine them. I measured three scenarios. With one slow instance, averaging the p99s was thirteen percent low and taking the max was six hundred eight percent high. With one instance carrying ninety percent of the traffic, averaging was eighty-five percent low and the max was two percent high. So which aggregation is less wrong flips depending on something you don't know. The fix is to send histograms instead of summaries, because bucket counts add. It's the same shape as unique-user counts – a summary can't be re-aggregated, so you ship a mergeable structure instead. Worth noting the merge operations differ: HyperLogLog merges by maximum because it records whether it saw something; histograms merge by sum because they record how many times. And the cost is real – sixty-four buckets gets the error under one percent and multiplies your series count by sixty-four, so I'd only do it for the metrics that carry an SLO."

15. Follow-ups – alerting, traces, exemplars

"Alerts ఈ data మీదే నడుస్తాయా?"

అవును, మరియు అది ఒక ముఖ్యమైన పరిణామం: మీ metrics system **ఒక availability భాగం** అవుతుంది. అది నెమ్మదైతే alerts ఆలస్యమవుతాయి; అది పడిపోతే మీకు **ఏమీ పడిపోలేదని** అనిపిస్తుంది. కాబట్టి రెండు నియమాలు: alerting queries **సరళంగా మరియు ముందే లెక్కించబడినవిగా** ఉండాలి (recording rules), మరియు monitoring system ని **ఒక వేరే monitoring system** చూడాలి (meta-monitoring).

"Prometheus బతికే ఉందా" అనే alert Prometheus నుంచి రాకూడదు.

"§4 lo నిషేధించిన labels – ఆ సమాచారం అవసరమైతే?"

అది logs మరియు traces పని, మరియు ఆ తేడాకి ఒక స్పష్టమైన నియమం ఉంది:

	ఏమి చెబుతుంది	cardinality
Metrics	"ఎంత, ఎంత తరచుగా" – కాలక్రమేణా	పరిమితంగా ఉండాలి
Logs	"ఈ ఒక్క సంఘటనలో ఏమి జరిగింది"	అపరిమితం, సరే
Traces	"ఈ ఒక్క request ఎక్కడ సమయం గడిపింది"	అపరిమితం, సరే

Metrics **ఏకరీతిగా కుంచించబడతాయి** కాబట్టి చౌకగా ఉంటాయి. Logs కుంచించబడవు కాబట్టి మీరు వాటిని **నమూనా (sample)** చేస్తారు. ఆ రెండు ఖర్చు నమూనాలు పూర్తిగా వేరు, మరియు ఒకదాన్ని ఇంకొకదానిగా వాడితే §4 జరుగుతుంది.

"ఒక శిఖరం కనిపించింది – ఏ request అది?"

ఇక్కడ **exemplars**: histogram bucket తో పాటు ఆ bucket lo పడిన **ఒక నిజమైన request యొక్క trace id** ని ఉంచడం. Cardinality పెరగదు (ఒక్క bucket కి ఒక్క exemplar), మరియు మీరు "p99 ఎందుకు పెరిగింది" నుంచి "ఆ నిర్దిష్ట request ని చూపించు" కి నేరుగా వెళ్ళగలరు. **నేను దీన్ని అమలు చేయలేదు** – కానీ ఇది metrics మరియు traces మధ్య ఒక వంతెన, మరియు అది §4 ని ఉల్లంఘించకుండా.

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు విషయాలు: **query performance** (నేను ingestion మరియు నిల్వ కొలిచాను; "గత 30 రోజుల 200 services ని 5 నిమిషాల ముద్దలుగా చూపించు" అనే query యొక్క ఖర్చు ఒక వేరే doc), **pull vs push** (నేను ఏ సేకరణ పద్ధతిన simulate చేయలేదు), మరియు **నిజమైన Gorilla నివృత్తి** – §7 lo చెప్పినట్టు నా 3.47 bytes నిరాశావాదమైనది, ఎందుకంటే నా కృత్రిమ series నిజమైన వాటి కంటే ఎక్కువ చలిస్తాయి.

16. ఏమి నేర్చుకున్నాం

1. ఈ problem lo scale సంఖ్య throughput కాదు – అది cardinality. సెకనుకి 56 MB ఒక్క machine. కానీ ఒక్క label **6 machines ని 567 చేస్తుంది.**

2. అత్యంత ఖరీదైన నిర్ణయం అత్యంత చౌకగా కనిపిస్తుంది. ఒక label జోడించడం ఒక్క పంక్తి మరియు దాని ఖర్చు ఒక వారం తర్వాత కనిపిస్తుంది. అందుకే పరిమితి **code lo** ఉండాలి, documentation lo కాదు.

3. సగటు ఒక కుంచింపు కాదు – అది ఒక తొలగింపు. 30 సెకన్ల శిఖరం 2,496 ms, 5 నిమిషాల సగటులో **281 ms.**

4. కొన్ని ప్రశ్నలకి జవాబులు తిరిగి లెక్కించలేరు. సగటు బతుకుతుంది, గరిష్ఠం (విడిగా ఉంచితే) బతుకుతుంది, **p99 బతకదు** – 60 నుంచి 278 లేదా 2,420.

5. Order statistics కూడబడవు. p99 ల సగటు **-85%**, p99 ల గరిష్ఠం **+608%**, మరియు **ఏది మెరుగ్గా పరిస్థితిని బట్టి మారుతుంది.**

6. సాధారణ రోజుల్లో సరిగ్గా ఉండే కొలత అత్యంత ప్రమాదకరమైనది. అన్నీ ఒకేలా ఉంటే p99 ల సగటు కేవలం 1% తప్పు – అంటే అది ఏదో తప్పు జరిగిన రోజున మాత్రమే అబద్ధం చెబుతుంది.

7. Merge ఆపరేషన్ ఆ నిర్మాణం ఏమి గుర్తుపెడుతుందో దాన్ని బట్టి ఉంటుంది. HLL గరిష్ఠం ("చూశానా"), histogram కూడిక ("ఎన్నిసార్లు"). తారుమారు చేస్తే 100/100 విఫలం.

8. "p99" అనే మాటకి ఒక నిర్వచనం లేదు. నా నియమం **300 lo 205** విఫలమైంది – code తప్పు కాదు, రెండు వేర్వేరు rank conventions.

9. ఒక mutation బతికితే మే data ని కూడా అనుమానించండి. నా నిరంతర యాదృచ్ఛిక విలువలు "సరిగ్గా bucket హద్దు మీద" అనే సందర్భాన్ని ఎప్పుడూ సృష్టించలేదు – నిజ జీవితంలో అది సర్వసాధారణం.

ఈ DOC నుంచి – మరియు ఈ 18 DOCS నుంచి – ఒక్క వాక్యం

ఈ series lo ప్రతి doc ఒకే పని చేసింది: ఒక సహజమైన design ని తీసుకుని, అది విరిగే వరకు నడిపి, ఆ విరుపుని ఒక సంఖ్యగా రాసింది.

ఆ సంఖ్యలే తేడా. "Last-write-wins డేటా పోగొడుతుంది" అనేది అందరికీ తెలుసు – "ఒక్కో cart కి సరిగ్గా 2.00 చేర్పులు" అనేది మీరు దాన్ని నడిపారని చెబుతుంది. "Retry storms చెడ్డవి" అనేది ఒక అభిప్రాయం – "99.1% వృథా, మరియు అమ్మకం ఒక్క tick కూడా వేగంగా ముగియలేదు" అనేది ఒక కొలత.

మరియు ప్రతిసారీ, ఆ సంఖ్యని వెతకడంలో నా సొంత తప్పు ఒకటి బయటపడింది – ఒక bug, ఒక తప్పు నియమం, లేదా ఒక mutation ఏమీ మార్చలేదని గ్రహించడం.

అదే ఈ పద్ధతి యొక్క నిజమైన విలువ: మీరు కొలిచే వరకు, మీకు తెలిసిందనుకున్నది నిజంగా తెలుసా అని మీకు తెలియదు.